

Universidad Europea de Madrid

Grado en Ingeniería Informática

Técnicas de Hacking — Curso 2025/2026

Práctica 3: MITM y Suplantación

Detección de ARP Spoofing y DNS Snooping mediante análisis pasivo de tráfico

Autor:

Sergio García

Profesor:

Alfredo Robledano Abasolo

Fecha:

Mayo 2026

1. Resumen

Se implementa un sistema de detección de intrusos para dos ataques de red: ARP Spoofing (envenenamiento de tablas ARP) y DNS Snooping (enumeración masiva de subdominios). Ambos se usan como fase previa a ataques más graves: el primero para interceptar tráfico en silencio, el segundo para envenenar la caché de un resolovedor DNS o cartografiar la infraestructura interna.

Los escenarios se reprodujeron en entornos Docker Compose aislados. El ataque ARP se ejecutó con bettercap; el tráfico DNS malicioso se generó con un script propio en Scapy.

Los dos detectores, también escritos en Scapy, funcionaron correctamente: `alert_arpspoof` identificó el cambio de MAC provocado por el envenenamiento ARP, y `alert_dnssnooping` detectó la ráfaga de consultas al superar el umbral configurado. Ninguno requirió infraestructura adicional más allá de análisis pasivo de tráfico.

Índice

1. Resumen	2
2. Introducción	5
3. Fundamentos teóricos	5
3.1. ARP Spoofing y ataques MITM	5
3.2. Ataque de Kaminsky y DNS Snooping	6
4. Desarrollo	6
4.1. Escenario de red para ARP Spoofing	6
4.2. Escenario de red para DNS Snooping	9
5. Resultados	10
5.1. Detección de ARP Spoofing	10
5.2. Detección de DNS Snooping	11
6. Conclusiones	13
7. Bibliografía	14
Bibliografía	14

Índice de figuras

Figura 1 Entorno Docker ARP levantado con los cuatro contenedores activos: attacker, router, victim y webserver.	7
Figura 2 Verificación de la instalación de bettercap v2.41.5 en el contenedor atacante. . .	7
Figura 3 bettercap en ejecución: el módulo arp.spoof detecta el endpoint víctima 192.168.10.10 e inicia el envenenamiento ARP.	8
Figura 4 Entorno Docker DNS con dnsserver (BIND9) y resolver (Alpine) activos en la red 172.20.0.0/24.	9
Figura 5 Salida de alert_arp spoof: detección del cambio de MAC en la IP 192.168.10.1 provocado por bettercap.	11
Figura 6 Script dns_attack.py completando el envío de 30 consultas DNS a subdominios aleatorios de example.com.	12
Figura 7 Alerta generada por alert_dnssnooping al superar el umbral de 10 consultas en 5 segundos desde 172.20.0.1.	13

2. Introducción

Los ataques de capa 2 pasan desapercibidos con más frecuencia de lo que debería, en parte porque la atención defensiva suele concentrarse en capas superiores. Sin monitorización activa del tráfico local, ARP Spoofing y DNS Snooping pueden operar durante tiempo indefinido sin dejar rastro visible para el usuario afectado.

El protocolo ARP (Address Resolution Protocol) fue diseñado en los años 80 sin mecanismos de autenticación [1]. Su funcionamiento es sencillo: cuando un nodo necesita conocer la dirección MAC asociada a una IP, emite una solicitud ARP en broadcast y el nodo correspondiente responde con su MAC. El problema es que cualquier nodo de la red puede enviar una respuesta ARP sin que nadie lo haya solicitado, y los sistemas operativos actualizan su caché sin verificar la legitimidad de esa respuesta. Esta carencia es la base del ARP Spoofing [2].

El ARP Spoofing permite a un atacante asociar su propia dirección MAC con la IP de otro nodo legítimo, normalmente el gateway de la red. A partir de ese momento, el tráfico destinado al gateway pasa primero por el atacante, que puede leerlo, modificarlo o descartarlo antes de reenviarlo. Este escenario se denomina ataque Man-In-The-Middle (MITM) y constituye una amenaza grave para la confidencialidad e integridad de las comunicaciones en redes locales.

Por otro lado, el sistema de nombres de dominio (DNS) también presenta vulnerabilidades conocidas. El ataque de Kaminsky [3], presentado públicamente en 2008, demostró cómo era posible envenenar la caché de un resolvidor DNS recursivo mediante una inyección masiva de respuestas falsificadas. La mecánica del ataque se apoya en la resolución de subdominios inexistentes: el atacante fuerza al resolvidor a hacer consultas recursivas y, mientras espera la respuesta del servidor autoritativo, intenta adivinar el identificador de transacción para inyectar una respuesta falsa antes que la legítima.

Relacionada con esta técnica, la denominada DNS Snooping [4] aprovecha el análisis de la caché de un resolvidor para deducir qué dominios han sido consultados recientemente desde la red interna, lo que permite cartografiar la infraestructura o los hábitos de navegación de los usuarios.

Esta práctica implementa dos firmas de detección basadas en análisis pasivo de tráfico para identificar ambos tipos de ataque en tiempo real, sin interferir con el tráfico legítimo.

3. Fundamentos teóricos

3.1. ARP Spoofing y ataques MITM

El protocolo ARP opera en la capa 2 del modelo OSI y es fundamental para el funcionamiento de cualquier red Ethernet. Cuando un equipo quiere comunicarse con una IP de su misma subred, consulta primero su tabla ARP local. Si no encuentra una entrada para esa IP, emite un paquete ARP Request en broadcast preguntando «¿quién tiene la IP X?». El equipo con

esa IP responde con un ARP Reply indicando su dirección MAC, y el solicitante almacena esa asociación en su caché ARP durante un tiempo determinado.

La vulnerabilidad surge porque los sistemas operativos aceptan ARP Replies no solicitadas (denominadas *gratuitous ARP*). Un atacante puede enviar continuamente respuestas ARP falsas indicando que su MAC corresponde a la IP del router, logrando que la víctima actualice su caché y empiece a enviarle el tráfico. Si simultáneamente hace lo mismo con el router (asociando su MAC a la IP de la víctima), se sitúa como intermediario entre ambos, configurando un ataque MITM completo.

La detección de este tipo de ataque se basa en monitorizar las respuestas ARP y detectar cuándo una IP conocida aparece asociada a una dirección MAC diferente a la que tenía registrada, lo que indica que alguien está intentando sobrecribir la caché de forma ilegítima.

3.2. Ataque de Kaminsky y DNS Snooping

El DNS es un sistema distribuido y jerárquico que traduce nombres de dominio en direcciones IP. Cuando un cliente consulta un nombre, su resolvidor local pregunta de forma recursiva a los servidores autoritativos hasta obtener la respuesta. Para reducir la carga de consultas repetidas, los resolvidores almacenan las respuestas en caché durante el tiempo indicado por el campo TTL.

El ataque de Kaminsky [3] explota este mecanismo de caché. El atacante genera una gran cantidad de consultas a subdominios aleatorios e inexistentes de un dominio objetivo (por ejemplo, `abcxyz.example.com`). Cada vez que el resolvidor hace la consulta recursiva correspondiente, el atacante intenta inyectar una respuesta DNS falsificada que incluya un registro NS malicioso para el dominio raíz. Si lo consigue antes de que llegue la respuesta legítima, envenena la caché del resolvidor para ese dominio.

La firma de detección implementada en esta práctica se basa en el análisis por volumen (*threshold*): una ráfaga de consultas DNS procedentes de un mismo origen hacia subdominios aleatorios en una ventana de tiempo corta es estadísticamente anómala y puede indicar la ejecución de este tipo de ataque.

4. Desarrollo

4.1. Escenario de red para ARP Spoofing

4.1.1. Diseño de la topología

El entorno de pruebas para la parte de ARP se ha definido mediante Docker Compose [5] con cuatro contenedores distribuidos en dos redes virtuales aisladas:

- **Red LAN** (192.168.10.0/24): red interna que simula la red local de una organización. Contiene el nodo víctima (192.168.10.10), el nodo atacante con bettercap (192.168.10.99) y el router (192.168.10.2).
- **Red WAN** (10.0.0.0/24): red externa que simula Internet o una DMZ. Contiene el router y el servidor web Nginx (10.0.0.10).

El router tiene una interfaz en cada red y actúa como pasarela con IP forwarding habilitado, lo que reproduce el comportamiento de un gateway real en una red corporativa. El servidor web actúa como destino legítimo al que la víctima accedería normalmente.

El nodo atacante utiliza la imagen oficial de Kali Linux y dispone de capacidades de red elevadas (NET_ADMIN y NET_RAW en Docker) para poder manipular y enviar paquetes a nivel de enlace, lo cual es necesario para que bettercap pueda operar correctamente.

```
(kali@kali)-[~/Laboratorio-hacking/P3_Mitm_y_suplantacion/src/arp]
$ docker compose down
docker compose up -d
docker compose ps
[+] Running 6/6
  ✓ Container router      Removed      0.1s
  ✓ Container webserver   Removed      0.5s
  ✓ Container victim      Removed     10.7s
  ✓ Container attacker    Removed     10.7s
  ✓ Network arp_lan       Removed      0.7s
  ✓ Network arp_wan       Removed      0.4s
[+] Running 6/6
  ✓ Network arp_wan       Created      0.3s
  ✓ Network arp_lan       Created      0.3s
  ✓ Container router      Started      2.4s
  ✓ Container webserver   Started      2.4s
  ✓ Container victim      Started      2.3s
  ✓ Container attacker    Started      2.1s
```

NAME	IMAGE	STATUS	PORTS	COMMAND	SERVICE	CR
attacker	kalilinux/kali-rolling	Up 1 second		"sleep infinity"	attacker	3
router	alpine	Up 1 second		"sh -c 'echo 1 > /pr..."	router	3
victim	alpine	Up 1 second		"sh -c 'apk add --no..."	victim	3
webserver	nginx	Up 1 second	80/tcp	"/docker-entrypoint..."	webserver	3

```
(kali@kali)-[~/Laboratorio-hacking/P3_Mitm_y_suplantacion/src/arp]
$
```

Figura 1: Entorno Docker ARP levantado con los cuatro contenedores activos: attacker, router, victim y webserver.

4.1.2. Herramienta de ataque: bettercap

Para simular el envenenamiento ARP se ha utilizado bettercap [2] (v2.41.5), una herramienta de auditoría de red de código abierto habitual en pruebas de penetración. Su módulo arp.spoof automatiza el envío continuo de respuestas ARP falsificadas hacia los objetivos configurados.

```
(kali@kali)-[~/Laboratorio-hacking/P3_Mitm_y_suplantacion/src/arp]
$ docker exec -it attacker bettercap --version
bettercap v2.41.5 (built for linux amd64 with go1.24.9)

(kali@kali)-[~/Laboratorio-hacking/P3_Mitm_y_suplantacion/src/arp]
$ |
```

Figura 2: Verificación de la instalación de bettercap v2.41.5 en el contenedor atacante.

El ataque se lanzó desde el interior del contenedor atacante especificando la víctima como objetivo y el router como gateway a suplantar:

```
bettercap -iface eth0 -eval \
  "set arp.spoof.targets 192.168.10.10; \
```

```
set arp.spoof.gateway 192.168.10.2; \
arp.spoof on; net.probe on"
```

El módulo `net.probe` se activa para que `bettercap` descubra los nodos activos en la red antes de iniciar el spoofing. Una vez identificada la víctima, el módulo `arp.spoof` empieza a enviar respuestas ARP falsificadas de forma periódica para mantener envenenadas las cachés ARP tanto de la víctima como del router.

```
(kali㉿kali)~[/.../Laboratorio-hacking/P3_Mitm_y_suplantacion/src/arp]
$ docker exec -it attacker bettercap -iface eth0 -eval "set arp.spoof.targets 192.168.10.10; set arp.spoof.gateway 192.168.10.2; arp.spoof on; net.probe on"
bettercap v2.41.5 (built for linux amd64 with go1.24.9) [type 'help' for a list of commands]

192.168.10.0/24 > 192.168.10.99 » [01:39:25] [sys.log] [inf] gateway monitor started ...
192.168.10.0/24 > 192.168.10.99 » [01:39:25] [sys.log] [inf] arp.spoof starting net.recon
as a requirement for arp.spoof
192.168.10.0/24 > 192.168.10.99 » [01:39:25] [sys.log] [inf] arp.spoof arp spoofer started
, probing 1 targets.
192.168.10.0/24 > 192.168.10.99 » [01:39:25] [sys.log] [inf] net.probe probing 256 address
es on 192.168.10.0/24
192.168.10.0/24 > 192.168.10.99 » [01:39:25] [sys.log] [inf] zerogod service discovery sta
rted
192.168.10.0/24 > 192.168.10.99 » [01:39:25] [sys.log] [war] arp.spoof could not find spoo
f targets
192.168.10.0/24 > 192.168.10.99 » [01:39:25] [endpoint.new] endpoint 192.168.10.10 detecte
d as 02:42:c0:a8:0a:0a.
192.168.10.0/24 > 192.168.10.99 » |
```

Figura 3: `bettercap` en ejecución: el módulo `arp.spoof` detecta el endpoint víctima 192.168.10.10 e inicia el envenenamiento ARP.

4.1.3. Función de detección: `alert_arp spoof`

La función de detección `alert_arp spoof` se ha implementado en Python utilizando la librería Scapy [6]. El script escucha el tráfico ARP en la interfaz bridge de la red LAN (`br-e031e5a89389`) y mantiene una tabla interna de asociaciones IP-MAC observadas. Cada vez que llega un paquete ARP de tipo Reply (código de operación 2), se extrae la IP y la MAC del campo origen y se comprueba si ya existe una entrada para esa IP en la tabla:

- Si no existe, se registra la asociación como legítima.
- Si existe y la MAC es diferente a la registrada, se genera una alerta indicando la IP afectada, la MAC original y la nueva MAC sospechosa.

La lógica central de la firma es la siguiente:

```
def alert_arp spoof(packet):
    if packet.haslayer(ARP) and packet[ARP].op == 2:
        ip_src = packet[ARP].psrc
        mac_src = packet[ARP].hwsrc
        if ip_src in arp_table:
            if arp_table[ip_src] != mac_src:
                print(f"[!] ARP SPOOFING DETECTADO")
                print(f"    IP      : {ip_src}")
                print(f"    MAC OK : {arp_table[ip_src]}")
                print(f"    MAC NEW: {mac_src} <-- SOSPECHOSA")
            else:
                arp_table[ip_src] = mac_src
```

El ARP Spoofing requiere enviar respuestas ARP no solicitadas (*gratuitous ARP*) de forma continua para mantener la caché de la víctima envenenada, lo que genera un flujo constante de paquetes que el detector puede analizar.

4.2. Escenario de red para DNS Snooping

4.2.1. Diseño de la topología

Para la parte de DNS se ha definido un entorno independiente con dos nodos en la red dnsnet (172.20.0.0/24):

- **dnsserver** (172.20.0.10): servidor DNS basado en BIND9 [7], configurado con recursividad habilitada y reenvío de consultas a 8.8.8.8. Actúa como resolutor recursivo, el objetivo típico de los ataques de Kaminsky.
- **resolver** (172.20.0.20): nodo cliente Alpine Linux que simula las peticiones DNS de un usuario o sistema interno.

La separación entre servidor DNS y resolver en nodos distintos permite reproducir de forma más fiel un escenario real, donde el resolutor es un servidor de la organización que recibe las consultas de los clientes internos y las resuelve de forma recursiva.

```
(kali@kali)~[~/Laboratorio-hacking/P3_Mitm_y_suplantacion/src/arp]
$ cd ~/Desktop/Laboratorio-hacking/P3_Mitm_y_suplantacion/src/dns
docker compose up -d
docker compose ps
[+] Running 14/14
  ✓ dnsserver Pulled
    ✓ 589002ba0eae Already exists 5.0s
    ✓ 20559944753a Pull complete 0.0s
    ✓ a4249c311ccd Pull complete 0.6s
    ✓ f69b69b28720 Pull complete 0.7s
    ✓ 1c7648c07dba Pull complete 1.8s
    ✓ 48ab531ad2da Pull complete 2.6s
    ✓ 9a50bcb71e67 Pull complete 2.6s
    ✓ 9ab9a1d19262 Pull complete 2.7s
    ✓ d125872ef80d Pull complete 3.0s
    ✓ f34311c690cb Pull complete 3.1s
    ✓ 0867ad7feac6 Pull complete 3.2s
    ✓ a740a593a671 Pull complete 3.2s
    ✓ 5f5097a53df0 Pull complete 3.3s
[+] Running 3/3
  ✓ Network dns_dnsnet Created 0.2s
  ✓ Container dnsserver Started 2.6s
  ✓ Container resolver Started 2.6s
```

NAME	PORTS	IMAGE	COMMAND	SERVICE	CREATED	STATUS
dnsserver		internetsystemsconsortium/bind9:9.18	"/usr/sbin/named -u _"	dnsserver	3 seconds ago	Up 2 secon

Figura 4: Entorno Docker DNS con dnsserver (BIND9) y resolver (Alpine) activos en la red 172.20.0.0/24.

4.2.2. Función de detección: alert_dnssnooping

La función `alert_dnssnooping` implementa un mecanismo de detección basado en umbral de volumen (*threshold*). El script escucha el tráfico UDP en el puerto 53 de la interfaz bridge de la red DNS (br-dafe511130b7) y contabiliza las consultas DNS de tipo Query (bit QR=0) originadas desde cada dirección IP en una ventana temporal deslizando de 5 segundos. Si una misma IP supera las 10 consultas en ese intervalo, se considera tráfico anómalo y se genera una alerta.

La ventana deslizando se implementa manteniendo una lista de timestamps por IP y descartando los que queden fuera del intervalo en cada nueva consulta:

```
query_count[ip_src] = [t for t in query_count[ip_src]
                        if now - t < WINDOW]
query_count[ip_src].append(now)

if len(query_count[ip_src]) >= THRESHOLD:
    print(f"[!] DNS SNOOPING DETECTADO desde {ip_src}")
    print(f"    Consultas en {WINDOW}s: {len(query_count[ip_src])}")
```

El uso de una ventana deslizante en lugar de un contador fijo permite detectar ráfagas que comiencen en cualquier momento, sin depender de intervalos predefinidos que podrían dejar pasar un ataque que se inicie justo entre dos ventanas consecutivas.

4.2.3. Script generador de tráfico

Para validar el funcionamiento del detector se ha implementado el script `dns_attack.py`, que simula el comportamiento de un atacante ejecutando la fase de enumeración del ataque de Kaminsky. El script genera 30 paquetes UDP dirigidos al puerto 53 del nodo resolver (172.20.0.20), cada uno con un nombre de subdominio aleatorio de ocho caracteres sobre el dominio `example.com`. Atacar el resolver recursivo y no el servidor autoritativo es precisamente la mecánica del ataque de Kaminsky: el objetivo es que el resolver haga las consultas recursivas mientras el atacante intenta inyectar respuestas falsas. Los subdominios se generan de forma completamente aleatoria para forzar esas consultas recursivas:

```
def random_subdomain(length=8):
    return ''.join(random.choices(string.ascii_lowercase, k=length))

for i in range(COUNT):
    sub = random_subdomain()
    qname = f"{sub}.{DOMAIN}"
    pkt = IP(dst=DNS_SERVER) / UDP(dport=53) / DNS(
        rd=1, qd=DNSQR(qname=qname))
    send(pkt, verbose=0)
```

El intervalo entre consultas se fija en 0.1 segundos, lo que garantiza que se supere el umbral de 10 consultas en 5 segundos y se active la alerta del detector.

5. Resultados

Ambos detectores generaron alertas en tiempo real sin falsos negativos en los escenarios evaluados.

5.1. Detección de ARP Spoofing

En el momento en que bettercap comenzó a emitir respuestas ARP falsificadas, el detector `alert_arp spoof` identificó de inmediato la anomalía. El sistema registró primero las asociaciones legítimas observadas durante el arranque del entorno y, al recibir una respuesta ARP que asociaba la IP del router (192.168.10.1) a una dirección MAC diferente, generó la alerta correspondiente.

```
(kali㉿kali)-[~]
└─$ cd ~/Desktop/Laboratorio-hacking/P3_Mitm_y_suplantacion/src
└─$ sudo python3 alert_arp spoof.py
[sudo] password for kali:
[*] Iniciando monitorización ARP... (Ctrl+C para detener)
[*] MAC registrada -> 192.168.10.10 = 02:42:c0:a8:0a:0a
[*] MAC registrada -> 192.168.10.1 = 02:42:c0:a8:0a:63

[!] [21:39:30] ARP SPOOFING DETECTADO
    IP      : 192.168.10.1
    MAC OK  : 02:42:c0:a8:0a:63
    MAC NEW: 02:42:51:b6:f9:2b  <-- SOSPECHOSA
    Posible ataque MITM en curso

[*] MAC registrada -> 192.168.10.99 = 02:42:c0:a8:0a:63
|
```

Figura 5: Salida de alert_arp spoof: detección del cambio de MAC en la IP 192.168.10.1 provocado por bettercap.

El análisis del paquete capturado revela el patrón característico del ataque: la MAC legítima del router (02:42:c0:a8:0a:63) fue sustituida por la MAC del contenedor atacante (02:42:51:b6:f9:2b). A partir de ese momento, todo el tráfico de la víctima destinado al router habría pasado por el atacante.

IP	MAC legítima	MAC sospechosa	Resultado
192.168.10.1 (router)	02:42:c0:a8:0a:63	02:42:51:b6:f9:2b	ARP Spoofing ✓
192.168.10.10 (víctima)	02:42:c0:a8:0a:0a	—	Sin anomalías
192.168.10.99 (atacante)	02:42:c0:a8:0a:63	—	Sin anomalías

Tabla 1: Resumen de asociaciones ARP observadas y anomalías detectadas durante el ataque.

5.2. Detección de DNS Snooping

El script dns_attack.py completó el envío de las 30 consultas en aproximadamente 3 segundos, superando holgadamente el umbral de 10 consultas en 5 segundos.

```

(kali㉿kali)~[~]
$ cd ~/Desktop/Laboratorio-hacking/P3_Mitm_y_suplantacion/src/dns
docker compose up -d
docker compose ps
[+] Running 2/2
  ✓ Container dnsserver Started 2.6s
  ✓ Container resolver Started 2.5s
NAME                IMAGE                                COMMAND
SERVICE            CREATED          STATUS          PORTS
dnsserver            internetsystemsconsortium/bind9:9.18 "/usr/sbin/named -u ..."
dnsserver            15 hours ago    Up 2 seconds    53/tcp, 443/tcp, 853/tcp, 953/tcp,
53/udp
resolver            alpine
resolver            15 hours ago    Up 2 seconds    "sh -c 'apk add --no..."

(kali㉿kali)~[~/../Laboratorio-hacking/P3_Mitm_y_suplantacion/src/dns]
$ nano ~/Desktop/Laboratorio-hacking/P3_Mitm_y_suplantacion/src/dns_attack
.py

(kali㉿kali)~[~/../Laboratorio-hacking/P3_Mitm_y_suplantacion/src/dns]
$ cd ~/Desktop/Laboratorio-hacking/P3_Mitm_y_suplantacion/src
$ sudo python3 dns_attack.py
[sudo] password for kali:
[*] Iniciando ataque DNS Snooping simulado -> 172.20.0.20
[*] Enviando 30 consultas a subdominios aleatorios de example.com

[01] Consulta: kiqfunhi.example.com
[02] Consulta: vjlyiolp.example.com
[03] Consulta: ljffdfqr.example.com
[04] Consulta: fzdnihyy.example.com
[05] Consulta: sniumsik.example.com
[06] Consulta: btsuwixx.example.com
[07] Consulta: uthyzzpl.example.com
[08] Consulta: nlyrtsgo.example.com
[09] Consulta: uvyzusbcb.example.com
[10] Consulta: gxqrubno.example.com
[11] Consulta: ezbhcfzn.example.com
[12] Consulta: irhohnvi.example.com
[13] Consulta: wtepkpvq.example.com
[14] Consulta: vgvbiber.example.com

```

Figura 6: Script dns_attack.py completando el envío de 30 consultas DNS a subdominios aleatorios de example.com.

El detector alert_dnssnooping identificó la ráfaga en tiempo real y generó la alerta al alcanzar el umbral, mostrando la IP origen, el número de consultas detectadas en la ventana temporal y el último subdominio consultado.

```

[*] [12:48:45] Consulta DNS: 172.20.0.1 -> ezbhcfzn.example.com (1/10)
[*] [12:48:45] Consulta DNS: 172.20.0.1 -> irhohnvi.example.com (2/10)
[*] [12:48:45] Consulta DNS: 172.20.0.1 -> wtepkpvq.example.com (3/10)
[*] [12:48:45] Consulta DNS: 172.20.0.1 -> vgvbiber.example.com (4/10)
[*] [12:48:45] Consulta DNS: 172.20.0.1 -> rheyjzkn.example.com (5/10)
[*] [12:48:45] Consulta DNS: 172.20.0.1 -> xizlnvme.example.com (6/10)
[*] [12:48:46] Consulta DNS: 172.20.0.1 -> czwglivb.example.com (7/10)
[*] [12:48:46] Consulta DNS: 172.20.0.1 -> lkyawowj.example.com (8/10)
[*] [12:48:46] Consulta DNS: 172.20.0.1 -> wlmruxga.example.com (9/10)
[*] [12:48:46] Consulta DNS: 172.20.0.1 -> ausucshz.example.com (10/10)

[!] [12:48:46] DNS SNOOPING DETECTADO
Origen : 172.20.0.1
Consultas en 5s: 10 (umbral: 10)
Último subdominio: ausucshz.example.com
Posible ataque de Kaminsky o enumeración DNS

[*] [12:48:46] Consulta DNS: 172.20.0.1 -> pbbqmiew.example.com (1/10)
[*] [12:48:46] Consulta DNS: 172.20.0.1 -> bkrwaaup.example.com (2/10)
[*] [12:48:46] Consulta DNS: 172.20.0.1 -> gajbjzen.example.com (3/10)
[*] [12:48:46] Consulta DNS: 172.20.0.1 -> gycfdapw.example.com (4/10)
[*] [12:48:47] Consulta DNS: 172.20.0.1 -> eutykeih.example.com (5/10)
[*] [12:48:47] Consulta DNS: 172.20.0.1 -> xptgnlqy.example.com (6/10)
[*] [12:48:47] Consulta DNS: 172.20.0.1 -> ycgrginc.example.com (7/10)
[*] [12:48:47] Consulta DNS: 172.20.0.1 -> mbqgyarr.example.com (8/10)
[*] [12:48:47] Consulta DNS: 172.20.0.1 -> yymrvxln.example.com (9/10)
[*] [12:48:47] Consulta DNS: 172.20.0.1 -> yabormqb.example.com (10/10)

[!] [12:48:47] DNS SNOOPING DETECTADO
Origen : 172.20.0.1
Consultas en 5s: 10 (umbral: 10)

```

Figura 7: Alerta generada por alert_dnssnooping al superar el umbral de 10 consultas en 5 segundos desde 172.20.0.1.

El detector identificó únicamente la IP origen del script de ataque (172.20.0.1) como fuente de la ráfaga, alcanzando el umbral de 10 consultas en menos de 5 segundos. El nodo resolver (172.20.0.20) recibió los paquetes y los procesó internamente, pero no generó tráfico suficiente hacia fuera para disparar la alerta. Esto confirma que dirigir el ataque al resolver recursivo, tal y como ocurre en el ataque de Kaminsky real, produce el patrón de detección esperado sin falsos positivos.

IP origen	Consultas en 5s	Umbral	Resultado
172.20.0.1 (script ataque)	10	10	DNS Snooping ✓
172.20.0.20 (resolver)	< 10	10	Sin anomalías
172.20.0.10 (dnsserver)	< 10	10	Sin anomalías

Tabla 2: Resumen de alertas DNS generadas durante la simulación del ataque de Kaminsky.

6. Conclusiones

Detectar ARP Spoofing y DNS Snooping no requiere herramientas comerciales. Con análisis pasivo de tráfico y dos criterios simples, cambio de MAC en ARP y umbral de consultas DNS, se obtienen alertas funcionales que de otro modo habrían pasado desapercibidas.

El detector de ARP respondió con el primer paquete falsificado. Su punto débil es que asume como legítima la primera asociación IP-MAC que observa: si el atacante llega antes, no hay

alerta. En producción esto se resuelve con una lista blanca estática de asociaciones conocidas o integrando el detector con un inventario de activos.

El umbral de DNS funcionó para el escenario simulado, pero cualquier umbral fijo requiere calibración previa. Añadir análisis de aleatoriedad estadística sobre los nombres de subdominio consultados reduciría los falsos positivos en redes con tráfico DNS elevado.

Docker fue la parte más práctica de todo esto: levantar una topología de cuatro nodos con dos redes virtuales tomó minutos, y desmontar el entorno no dejó rastro. Describir la topología como `docker-compose.yml` también funciona como documentación.

Un paso siguiente lógico sería correlacionar ambos detectores: el ARP Spoofing se usa a menudo como paso previo para interceptar y manipular el tráfico DNS, forzando la resolución de dominios maliciosos. Detectarlos juntos daría más señal que detectarlos por separado.

7. Bibliografía

Bibliografía

- [1] D. C. Plummer, «An Ethernet Address Resolution Protocol», RFC 826, 1982.
- [2] S. Margaritelli, «bettercap: The Swiss Army Knife for 802.11, BLE, IPv4 and IPv6 Networks». 2024.
- [3] D. Kaminsky, «It's the End of the Cache As We Know It», *Black Hat USA*, 2008.
- [4] C. Rodrigues, «DNS Snooping: Measuring the Exposure of Confidential Information via DNS», InfoSec Reading Room, 2003.
- [5] Docker Inc., «Docker Compose: Define and run multi-container applications». 2024.
- [6] P. Biondi y the Scapy community, «Scapy: Packet crafting for Python2 and Python3». 2024.
- [7] Internet Systems Consortium, «BIND 9: The most widely used DNS software on the Internet». 2024.